PES UNIVERSITY
100 feet Ring Road, BSK 3rd Stage, Bengaluru 560085

Department of Computer Science and Engineering
(AI & ML)
B. Tech. CSE - 5th Semester
Aug – Dec 2026

UE23CS351A
DATABASE MANAGEMENT SYSTEMS (DBMS)

PROJECT REPORT on

Vicinia: Hyperlocal Secondhand Marketplace using PostgreSQL/PostGIS and Neo4j

Submitted by:

SRN : PES1UG24AM276	Siri Krishna C R	5,E
SRN : PES1UG24AM279	Spoorthi Poornachnadra	5,E

Class of Prof. MIR JUNAID RASOOL

Abstract

Vicinia is a marketplace for secondhand goods such as furniture and electronics, where the location of an item matters as much as its price. The system uses PostgreSQL with PostGIS to store listings, orders and reviews and to search within a radius using a spatial index. A purchase is protected by a row lock and a unique constraint, so one item is never sold to two buyers.

Neo4j stores the trust, purchase and review relationships between users. It is used to compute a trust score for each seller and to detect groups of sellers who trade only among themselves. The score is written back to PostgreSQL and changes the search ranking.

The interface and the API are written in Next.js with login and three roles: buyer, seller and admin. This report contains the ER model, the relational schema, the SQL and Cypher statements, the results and the project structure.

Table of Contents

Sl. No	Topic	Page No.
1.	Introduction a) Problem Statement	4
2.	User Requirements Specification- Tech Stack	5
3.	E-R Model a) List of Entities [labeling Weak ones] b) List non-simple Attributes c) ER Diagram Snapshot d) ERD from tool like draw.io	7
4.	Relational Model a) List of Relations that are not Entities in ER Model b) Schema Diagram [ensure light background image]	11
5.	SQL DDL Statements	13
6.	SQL DML Statements	16
7.	NoSQL/ Vector Database Commands	22

8.	Results a) Resulting Tables' Screenshots	25
9.	Next.js-Project Tree Structure	26
10.	Few sample Screenshots of application code and demo	28
11.	Evaluation Rubrics- Completion Status	29

Introduction

a) Problem Statement

Online classifieds for secondhand goods treat location as a filter. For items like sofas, wardrobes and refrigerators the buyer wants someone nearby, and also wants to know whether that seller can be trusted. An ordinary shop database does not handle two things well here.

The first is finding listings within some distance of the buyer, sorted by distance, price and seller quality, while making sure that two buyers cannot both buy the same one-off item. The second is judging the seller. A star average is easy to fake: a seller can ask friends to buy and review. Trust between people is a relationship, so it is better stored as a graph.

Vicinia is a marketplace built for these two problems. PostgreSQL with PostGIS stores listings, orders and reviews and does the radius search and the safe purchase. Neo4j stores who trusts whom, who bought from whom and who reviewed whom, and is used to compute a trust score and to find groups of sellers who only trade among themselves. The score is written back to PostgreSQL and changes the search ranking.

b) Short Description

The application has three roles: buyer, seller and admin. Buyers search nearby listings, buy, return and review items, and mark sellers as trusted. Sellers manage listings, mark orders delivered and see a pricing hint. The admin runs the collusion scan and can take down the listings of a flagged seller. The interface and the API are built in Next.js. The schema has 10 PostgreSQL tables, 6 triggers, 1 procedure, several functions, 3 views and 1 materialised view, and a Neo4j graph with User nodes and TRUSTS, BOUGHT_FROM and REVIEWED relationships.

User Requirements Specification

a) Requirements

- Any user: register as buyer or seller and log in. Admin accounts cannot be self-registered.
- Buyer: search active listings within a radius of up to 50 km with category, price, minimum seller rating and text filters; sort by price, distance or a blended rank; view listing details with a delivery estimate; buy a listing (only one of two simultaneous buyers succeeds); return a delivered order and review it; mark a seller as trusted with a strength from 1 to 5; get recommendations that skip flagged sellers.
- Seller: create and manage own listings, see a dashboard with score card and orders, mark an order delivered, see a pricing hint of raising or cutting the price by 10 percent.
- Admin: run the collusion scan, see the leaderboard and flagged sellers, take down all active listings of a seller.
- System: recompute the seller score when reviews or order statuses change; reject a review unless the reviewer is the buyer of a delivered order.

b) Tech Stack and Justification

Technology	Used for	Why this stack
PostgreSQL	Users, listings, orders, reviews, scores	Constraints, ACID transactions and row-level locks keep ownership and money correct.
PostGIS (GiST index)	Locations of listings and users, delivery zones	A radius query uses a spatial index instead of computing the distance to every row. The geography type gives distances in metres.
Neo4j	Trust, purchase and review graph	Multi-hop trust paths and closed buying cycles are a single MATCH pattern. In SQL they need recursive queries and many self joins.
Next.js	Pages and API routes	One codebase for the role based pages and the API between the pages and both databases.
bcrypt, JWT cookie	Authentication	Passwords are stored as salted hashes. The session is a signed httpOnly cookie that page scripts cannot read.
zod	Input validation	Bad input is rejected with HTTP 400 before it reaches the database.

c) Choice of NoSQL Database

Neo4j was chosen because the trust problem is about paths and cycles between users. Personal trust follows up to three TRUSTS hops from the buyer to the seller. A collusion ring is a closed loop of BOUGHT_FROM relationships with no buyer from outside the loop. Both are short Cypher patterns, while in PostgreSQL they would need recursive CTEs or repeated self joins on the orders

table. A document or key-value store would not help, since the data has no natural document shape and the value is in the connections.

PostgreSQL remains the source of truth. After an order or review is committed the graph is updated on a best effort basis, so a Neo4j failure never blocks a sale. Neo4j sends the trust value and the flag back through the function apply_graph_signals.

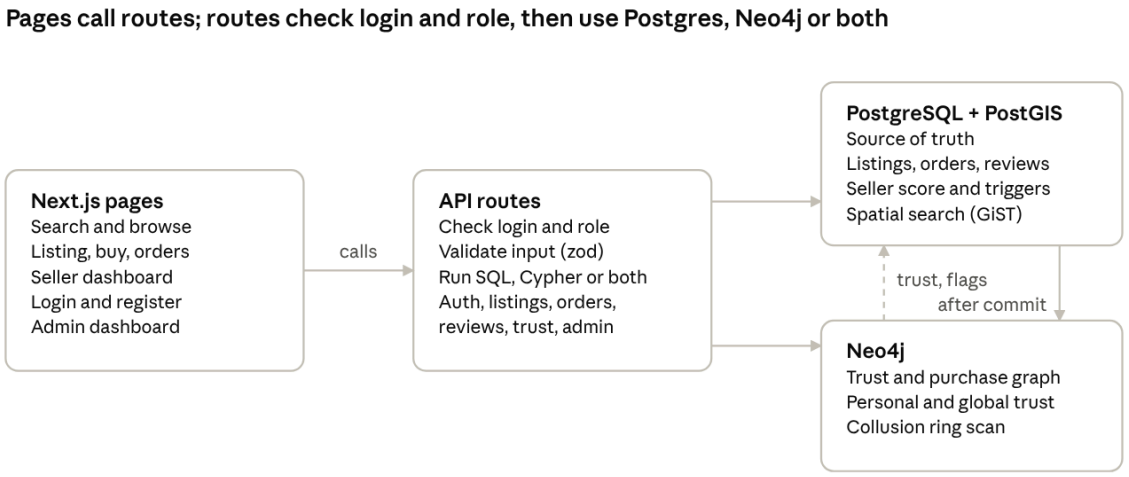


Figure 1: System architecture

d) Security

- Passwords are stored only as bcrypt hashes. Login gives the same 401 for an unknown email and a wrong password.
- The session is a JWT (HS256, 7 days) in an httpOnly cookie. The secret is read from the JWT_SECRET environment variable and .env is not committed.
- requireUser(roles) guards every protected route: 401 for not logged in, 403 for the wrong role. Registration accepts only buyer or seller.
- Inputs are checked by zod in the route and again by CHECK constraints. The review_integrity trigger stops forged reviews even if the API is called directly.

E-R Model

a) List of Entities

Entity	Attributes (key underlined in the schema)	Type
users	user_id, name, email, password_hash, role, location, created_at, phone	Strong
categories	category_id, name, parent_id	Strong
delivery_zones	zone_id, name, boundary, base_hours, per_km_hours	Strong
listings	listing_id, title, description, price, condition, status, location, created_at, search_vec, view_count	Strong
orders	order_id, status, total_amount, est_hours, placed_at, delivered_at	Strong
order_items	unit_price (identified by order_id and listing_id)	Weak (depends on orders)
price_history	history_id, old_price, new_price, changed_at	Strong
reviews	review_id, rating, comment, created_at	Strong
messages	message_id, body, sent_at	Strong
seller_metrics	avg_rating, review_count, orders_total, orders_returned, avg_response_mins, trust_boost, seller_score, is_flagged, updated_at (identified by the seller)	Strong

order_items and seller_metrics are weak entities. A row of order_items exists only inside an order and has no key of its own. A row of seller_metrics exists only for one seller and takes its key from users.

b) Non-simple Attributes

Attribute	Entity	Kind	Note
location	users, listings	Composite	Point with longitude and latitude, stored as GEOGRAPHY(Point, 4326)
boundary	delivery_zones	Composite	Polygon made of many points, stored as GEOGRAPHY(Polygon, 4326)
search_vec	listings	Derived	Generated from title and description
total_amount	orders	Derived	Sum of the unit prices of the items in the order
est_hours	orders	Derived	Zone base hours plus per-km hours times distance
avg_rating, review_count, orders_total, orders_returned, seller_score	seller_metrics	Derived	Recomputed from reviews and orders by triggers. Stored on purpose so that search stays fast

There are no multivalued attributes. Roles, listing status and order status are enumerated types.

c) ER Diagram Snapshot

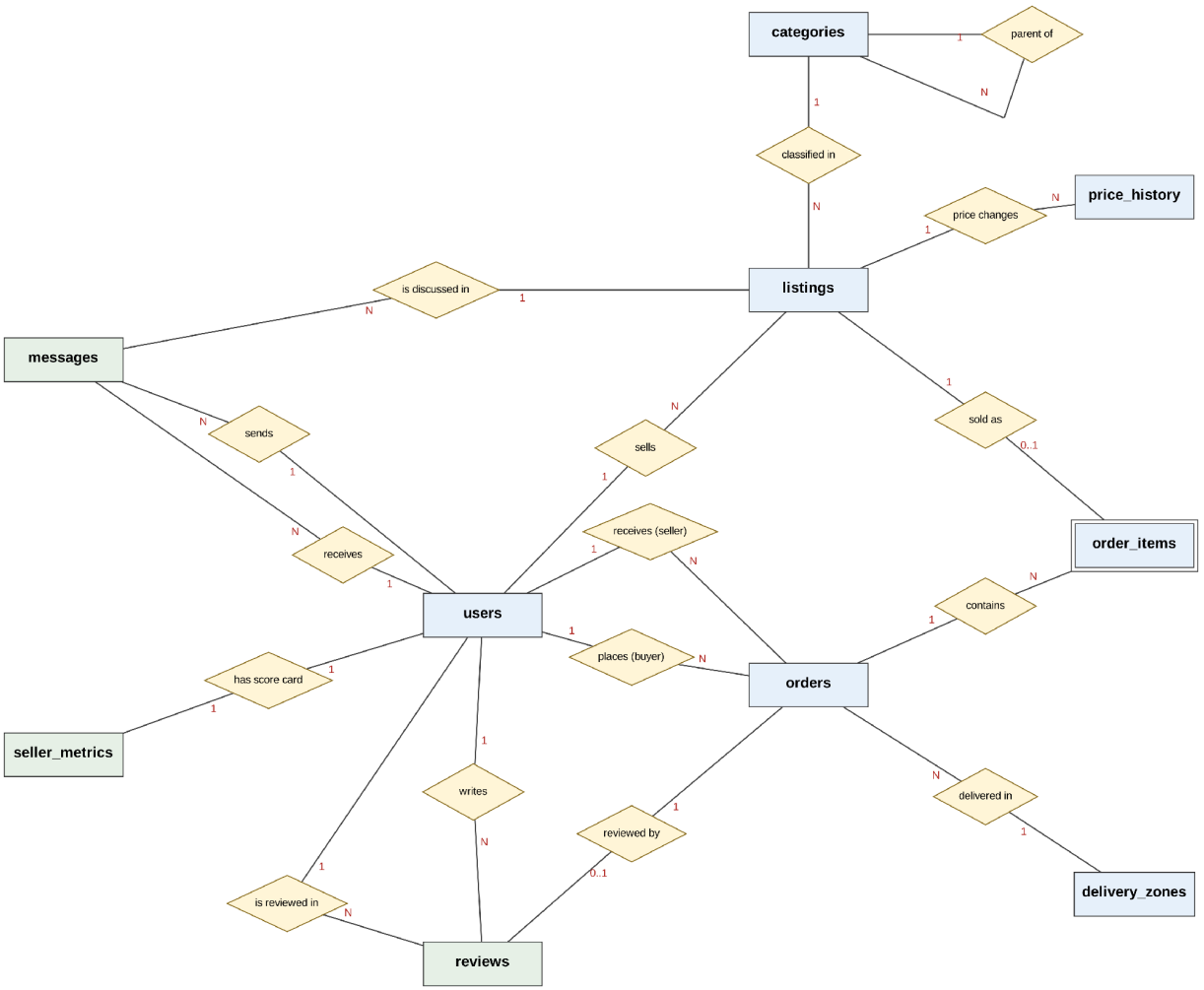


Figure 2: Conceptual ER diagram (rectangles are entities, diamonds are relationships, the double rectangle is the weak entity)

d) ERD from a Tool

The diagram below was drawn with Graphviz from the schema in 01_schema.sql. It shows every attribute, with PK, FK and UQ marks, and crow's foot notation for the cardinality.

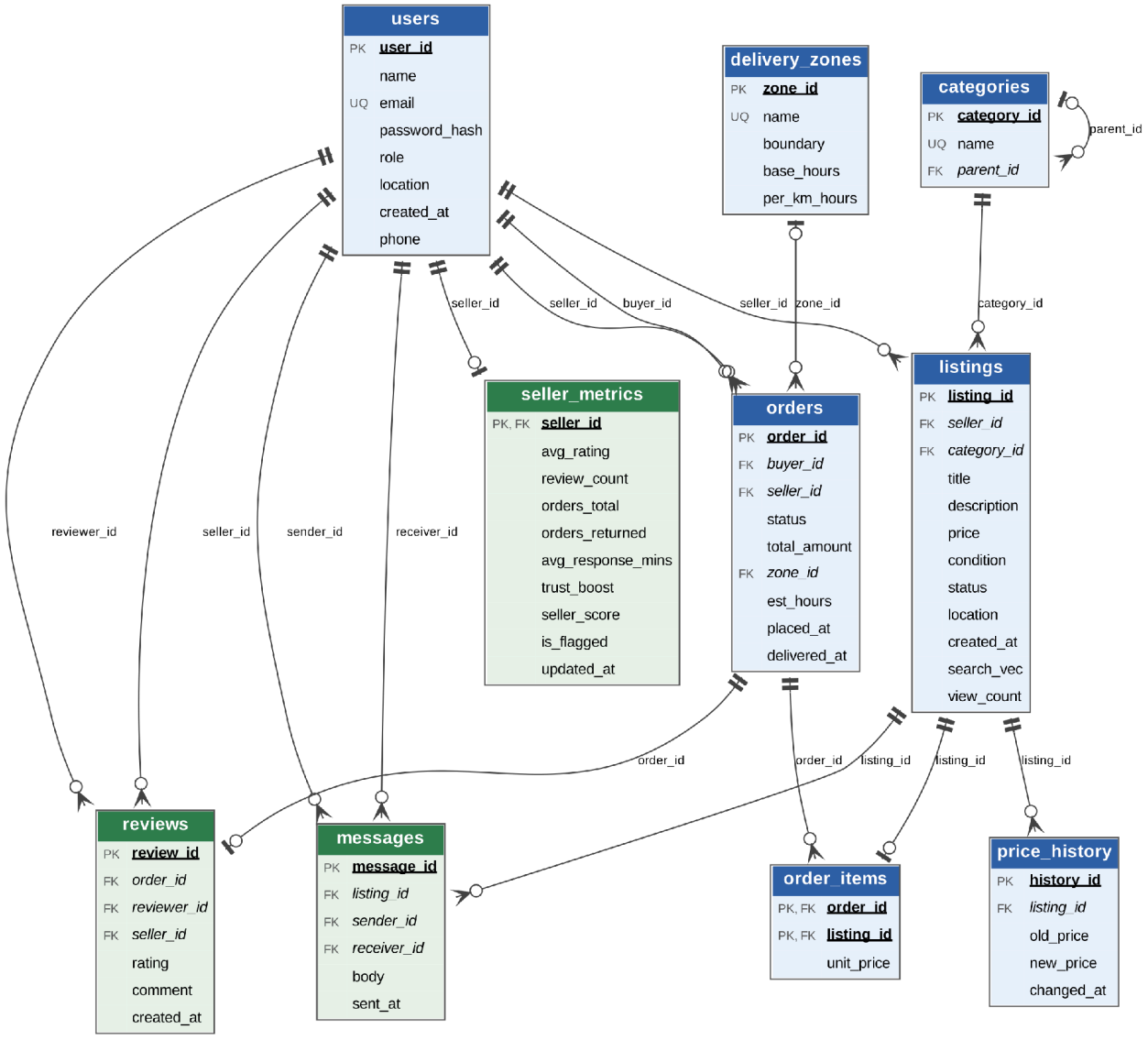


Figure 3: ER diagram with attributes and cardinalities

Relationship	One side	Many side	Cardinality	Foreign key
sells	users	listings	1 : N	listings.seller_id
classified in	categories	listings	1 : N	listings.category_id
parent of	categories	categories	1 : N	categories.parent_id
places (buyer)	users	orders	1 : N	orders.buyer_id
receives (seller)	users	orders	1 : N	orders.seller_id
delivered in	delivery_zones	orders	1 : N	orders.zone_id
contains	orders	order_items	1 : N	order_items.order_id

Relationship	One side	Many side	Cardinality	Foreign key
sold as	listings	order_items	1 : 0..1	order_items.listing_id (unique)
reviewed by	orders	reviews	1 : 0..1	reviews.order_id (unique)
writes	users	reviews	1 : N	reviews.reviewer_id
is reviewed in	users	reviews	1 : N	reviews.seller_id
is discussed in	listings	messages	1 : N	messages.listing_id
sends	users	messages	1 : N	messages.sender_id
receives	users	messages	1 : N	messages.receiver_id
price changes	listings	price_history	1 : N	price_history.listing_id
has score card	users	seller_metrics	1 : 0..1	seller_metrics.seller_id

Relational Model

a) List of Relations

Primary key attributes are underlined, foreign key attributes are in italics. Nine relations come directly from entities. order_items is the one that is not an entity of its own: it comes from the many to many relationship between orders and listings and carries unit_price. Its key is the pair (order_id, listing_id) and listing_id is also unique, so an item can be sold only once.

Relation	Attributes	Comes from
users	**user_id**, name, email, password_hash, role, location, created_at, phone	Entity
categories	**category_id**, name, *parent_id*	Entity
delivery_zones	**zone_id**, name, boundary, base_hours, per_km_hours	Entity
listings	**listing_id**, *seller_id*, *category_id*, title, description, price, condition, status, location, created_at, search_vec, view_count	Entity
orders	**order_id**, *buyer_id*, *seller_id*, status, total_amount, *zone_id*, est_hours, placed_at, delivered_at	Entity
order_items	***order_id***, ***listing_id***, unit_price	M:N relationship orders and listings (not an entity)
price_history	**history_id**, *listing_id*, old_price, new_price, changed_at	Entity
reviews	**review_id**, *order_id*, *reviewer_id*, *seller_id*, rating, comment, created_at	Entity
messages	**message_id**, *listing_id*, *sender_id*, *receiver_id*, body, sent_at	Entity
seller_metrics	***seller_id***, avg_rating, review_count, orders_total, orders_returned, avg_response_mins, trust_boost, seller_score, is_flagged, updated_at	Weak entity

Foreign keys: listings(seller_id) and orders(buyer_id, seller_id) and reviews(reviewer_id, seller_id) and messages(sender_id, receiver_id) refer to users. listings(category_id) and categories(parent_id) refer to categories. orders(zone_id) refers to delivery_zones. order_items(order_id) and reviews(order_id) refer to orders. order_items(listing_id), price_history(listing_id) and messages(listing_id) refer to listings. seller_metrics(seller_id) refers to users. Deleting a user, listing or order removes the rows that depend on it (ON DELETE CASCADE), except that deleting a parent category sets parent_id to NULL.

b) Schema Diagram

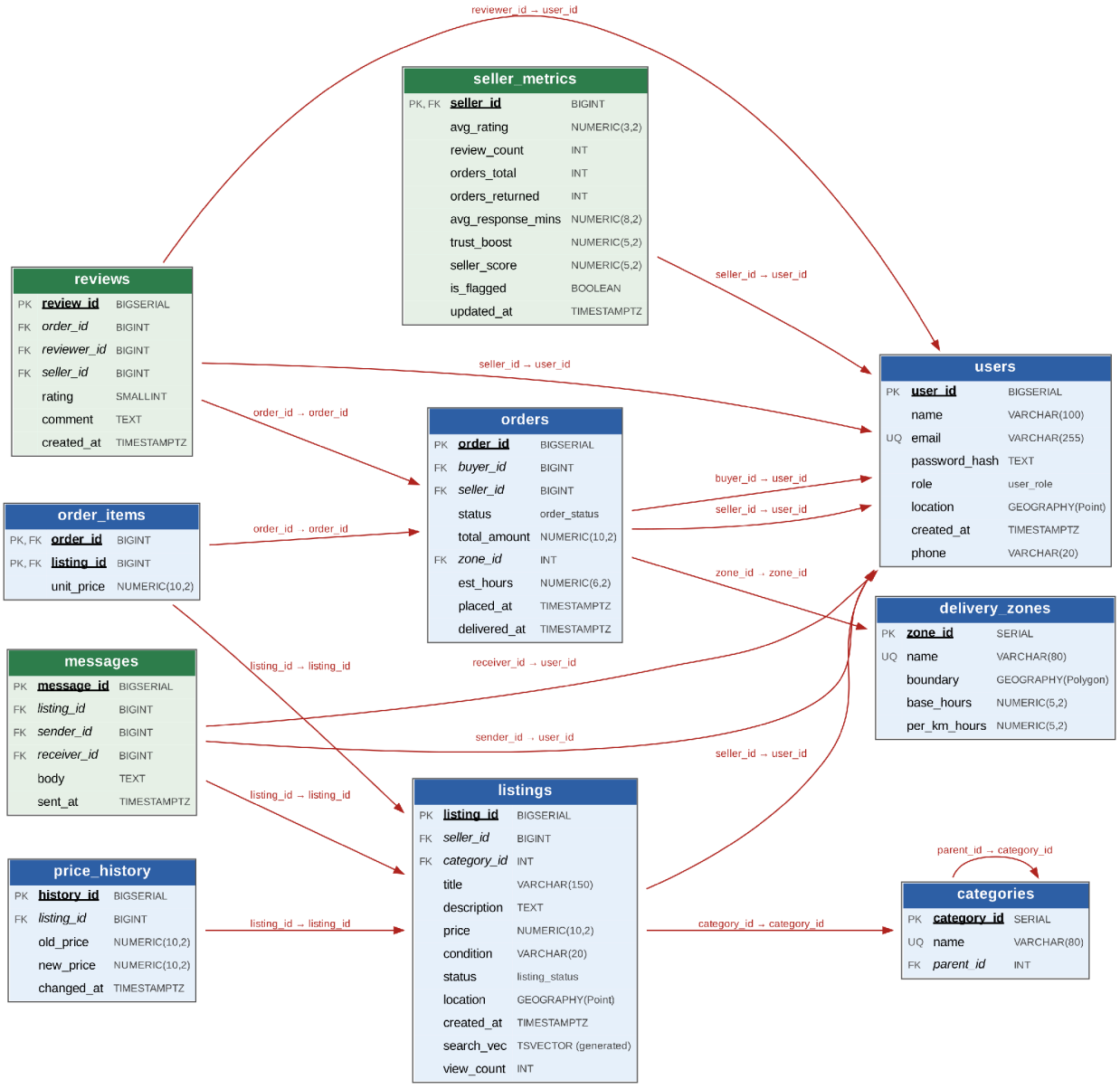


Figure 4: Relational schema diagram with data types and foreign key references

SQL DDL Statements

File: db/postgres/01_schema.sql. It enables PostGIS, creates the enumerated types, the tables with their constraints, the indexes (GiST, GIN and a partial index), four ALTER TABLE statements, three views and one materialised view. The DROP statements at the top let the file be run again from a clean state.

```
-- Vicinia :: PostgreSQL + PostGIS schema (DDL)
-- Run order: 01_schema.sql -> 02_functions.sql -> 03_seed.sql

CREATE EXTENSION IF NOT EXISTS postgis;
CREATE EXTENSION IF NOT EXISTS pgcrypto;   -- crypt()/gen_salt() for seed password hashes

-- Clean re-run support (demonstrates DROP)
DROP MATERIALIZED VIEW IF EXISTS mv_demand_by_cell CASCADE;
DROP VIEW IF EXISTS v_listing_search CASCADE;
DROP VIEW IF EXISTS v_order_summary CASCADE;
DROP VIEW IF EXISTS v_seller_leaderboard CASCADE;
DROP TABLE IF EXISTS price_history CASCADE;
DROP TABLE IF EXISTS messages CASCADE;
DROP TABLE IF EXISTS reviews CASCADE;
DROP TABLE IF EXISTS order_items CASCADE;
DROP TABLE IF EXISTS orders CASCADE;
DROP TABLE IF EXISTS listings CASCADE;
DROP TABLE IF EXISTS seller_metrics CASCADE;
DROP TABLE IF EXISTS delivery_zones CASCADE;
DROP TABLE IF EXISTS categories CASCADE;
DROP TABLE IF EXISTS users CASCADE;
DROP TYPE IF EXISTS user_role CASCADE;
DROP TYPE IF EXISTS listing_status CASCADE;
DROP TYPE IF EXISTS order_status CASCADE;

-- Enumerated types
CREATE TYPE user_role      AS ENUM ('buyer', 'seller', 'admin');
CREATE TYPE listing_status AS ENUM ('active', 'reserved', 'sold', 'removed');
CREATE TYPE order_status   AS ENUM ('placed', 'delivered', 'returned', 'cancelled');

-- users  (location is a geography point -> distances in metres)
CREATE TABLE users (
    user_id        BIGSERIAL PRIMARY KEY,
    name           VARCHAR(100) NOT NULL CHECK (length(trim(name)) >= 2),
    email          VARCHAR(255) NOT NULL UNIQUE
                       CHECK (email ~* '^[^@\s]+@[^@\s]+\.[^@\s]+$'),
    password_hash  TEXT         NOT NULL,                  -- bcrypt hash, never plain text
    role           user_role    NOT NULL DEFAULT 'buyer',
    location       GEOGRAPHY(Point, 4326) NOT NULL,
    created_at     TIMESTAMPTZ  NOT NULL DEFAULT now()
);
CREATE INDEX idx_users_location ON users USING GIST (location);

-- categories (self-referencing hierarchy)
CREATE TABLE categories (
    category_id  SERIAL PRIMARY KEY,
    name         VARCHAR(80) NOT NULL UNIQUE,
    parent_id    INT REFERENCES categories(category_id) ON DELETE SET NULL
);

-- delivery_zones (polygon per zone, used for delivery-time estimation)
CREATE TABLE delivery_zones (
    zone_id        SERIAL PRIMARY KEY,
    name           VARCHAR(80) NOT NULL UNIQUE,
    boundary       GEOGRAPHY(Polygon, 4326) NOT NULL,
    base_hours     NUMERIC(5,2) NOT NULL CHECK (base_hours > 0),
    per_km_hours   NUMERIC(5,2) NOT NULL DEFAULT 0.15 CHECK (per_km_hours >= 0)
);
CREATE INDEX idx_zones_boundary ON delivery_zones USING GIST (boundary);

-- listings
CREATE TABLE listings (
    listing_id   BIGSERIAL PRIMARY KEY,
    seller_id    BIGINT NOT NULL REFERENCES users(user_id) ON DELETE CASCADE,
    category_id  INT    NOT NULL REFERENCES categories(category_id),
    title        VARCHAR(150) NOT NULL CHECK (length(trim(title)) >= 3),
```

```
    description  TEXT NOT NULL DEFAULT '',
    price        NUMERIC(10,2) NOT NULL CHECK (price > 0),
    condition    VARCHAR(20) NOT NULL DEFAULT 'used'
                     CHECK (condition IN ('new','like_new','used','refurbished')),
    status       listing_status NOT NULL DEFAULT 'active',
    location     GEOGRAPHY(Point, 4326) NOT NULL,
    created_at   TIMESTAMPTZ NOT NULL DEFAULT now(),
    search_vec   TSVECTOR GENERATED ALWAYS AS
                     (to_tsvector('english', coalesce(title,'') || ' ' || coalesce(description,''))) STORED
);
-- Spatial index: makes ST_DWithin / KNN (<->) use an index instead of full scan
CREATE INDEX idx_listings_location ON listings USING GIST (location);
-- Full-text search
CREATE INDEX idx_listings_fts      ON listings USING GIN (search_vec);
-- Partial index: the hot path only ever searches active listings
CREATE INDEX idx_listings_active_cat_price ON listings (category_id, price) WHERE status = 'active';
CREATE INDEX idx_listings_seller   ON listings (seller_id);

-- orders / order_items
CREATE TABLE orders (
    order_id      BIGSERIAL PRIMARY KEY,
    buyer_id      BIGINT NOT NULL REFERENCES users(user_id),
    seller_id     BIGINT NOT NULL REFERENCES users(user_id),
    status        order_status NOT NULL DEFAULT 'placed',
    total_amount  NUMERIC(10,2) NOT NULL CHECK (total_amount > 0),
    zone_id       INT REFERENCES delivery_zones(zone_id),
    est_hours     NUMERIC(6,2),
    placed_at     TIMESTAMPTZ NOT NULL DEFAULT now(),
    CHECK (buyer_id <> seller_id)           -- cannot buy your own listing
);
CREATE INDEX idx_orders_buyer  ON orders (buyer_id);
CREATE INDEX idx_orders_seller ON orders (seller_id, status);

CREATE TABLE order_items (
    order_id    BIGINT NOT NULL REFERENCES orders(order_id) ON DELETE CASCADE,
    listing_id  BIGINT NOT NULL REFERENCES listings(listing_id),
    unit_price  NUMERIC(10,2) NOT NULL CHECK (unit_price > 0),
    PRIMARY KEY (order_id, listing_id),
    UNIQUE (listing_id)                      -- a one-off secondhand item can be sold only ONCE
);

-- reviews (one review per order)
CREATE TABLE reviews (
    review_id    BIGSERIAL PRIMARY KEY,
    order_id     BIGINT NOT NULL UNIQUE REFERENCES orders(order_id) ON DELETE CASCADE,
    reviewer_id  BIGINT NOT NULL REFERENCES users(user_id),
    seller_id    BIGINT NOT NULL REFERENCES users(user_id),
    rating       SMALLINT NOT NULL CHECK (rating BETWEEN 1 AND 5),
    comment      TEXT,
    created_at   TIMESTAMPTZ NOT NULL DEFAULT now()
);
CREATE INDEX idx_reviews_seller ON reviews (seller_id);

-- messages (used to compute seller response time)
CREATE TABLE messages (
    message_id   BIGSERIAL PRIMARY KEY,
    listing_id   BIGINT NOT NULL REFERENCES listings(listing_id) ON DELETE CASCADE,
    sender_id    BIGINT NOT NULL REFERENCES users(user_id),
    receiver_id  BIGINT NOT NULL REFERENCES users(user_id),
    body         TEXT NOT NULL CHECK (length(body) > 0),
    sent_at      TIMESTAMPTZ NOT NULL DEFAULT now(),
    CHECK (sender_id <> receiver_id)
);
CREATE INDEX idx_messages_listing ON messages (listing_id, sent_at);

-- seller_metrics (denormalised, maintained by triggers/functions)
CREATE TABLE seller_metrics (
    seller_id          BIGINT PRIMARY KEY REFERENCES users(user_id) ON DELETE CASCADE,
    avg_rating         NUMERIC(3,2)  NOT NULL DEFAULT 0 CHECK (avg_rating BETWEEN 0 AND 5),
    review_count       INT           NOT NULL DEFAULT 0 CHECK (review_count >= 0),
    orders_total       INT           NOT NULL DEFAULT 0 CHECK (orders_total >= 0),
    orders_returned    INT           NOT NULL DEFAULT 0 CHECK (orders_returned >= 0),
    avg_response_mins  NUMERIC(8,2)  NOT NULL DEFAULT 0 CHECK (avg_response_mins >= 0),
    trust_boost        NUMERIC(5,2)  NOT NULL DEFAULT 0,       -- pushed in from Neo4j (0..100)
    seller_score       NUMERIC(5,2)  NOT NULL DEFAULT 50 CHECK (seller_score BETWEEN 0 AND 100),
    is_flagged         BOOLEAN       NOT NULL DEFAULT FALSE,   -- set by collusion detection
    updated_at         TIMESTAMPTZ   NOT NULL DEFAULT now(),
```

```
    CHECK (orders_returned <= orders_total)
);

-- price_history (feeds dynamic pricing hints)
CREATE TABLE price_history (
    history_id   BIGSERIAL PRIMARY KEY,
    listing_id   BIGINT NOT NULL REFERENCES listings(listing_id) ON DELETE CASCADE,
    old_price    NUMERIC(10,2) NOT NULL,
    new_price    NUMERIC(10,2) NOT NULL,
    changed_at   TIMESTAMPTZ NOT NULL DEFAULT now()
);

-- ALTER examples (schema evolution)
ALTER TABLE users    ADD COLUMN phone VARCHAR(20);
ALTER TABLE users    ADD CONSTRAINT chk_phone CHECK (phone IS NULL OR phone ~ '^[0-9+\- ]{7,20}$');
ALTER TABLE listings ADD COLUMN view_count INT NOT NULL DEFAULT 0 CHECK (view_count >= 0);
ALTER TABLE orders   ADD COLUMN delivered_at TIMESTAMPTZ;

-- VIEWS

-- Search view: listing + seller score + seller location (ranking uses score)
CREATE VIEW v_listing_search AS
SELECT  l.listing_id, l.title, l.description, l.price, l.condition, l.status,
        l.category_id, c.name AS category, l.location, l.created_at, l.search_vec,
        l.seller_id, u.name AS seller_name,
        COALESCE(m.seller_score, 50) AS seller_score,
        COALESCE(m.avg_rating, 0)    AS avg_rating,
        COALESCE(m.is_flagged, FALSE) AS seller_flagged
FROM listings l
JOIN users u            ON u.user_id = l.seller_id
JOIN categories c       ON c.category_id = l.category_id
LEFT JOIN seller_metrics m ON m.seller_id = l.seller_id;

-- Order summary view
CREATE VIEW v_order_summary AS
SELECT  o.order_id, o.status, o.total_amount, o.placed_at, o.est_hours,
        b.user_id AS buyer_id,  b.name AS buyer_name,
        s.user_id AS seller_id, s.name AS seller_name,
        string_agg(l.title, ', ' ORDER BY l.title) AS items
FROM orders o
JOIN users b ON b.user_id = o.buyer_id
JOIN users s ON s.user_id = o.seller_id
JOIN order_items oi ON oi.order_id = o.order_id
JOIN listings l ON l.listing_id = oi.listing_id
GROUP BY o.order_id, b.user_id, b.name, s.user_id, s.name;

-- Seller leaderboard
CREATE VIEW v_seller_leaderboard AS
SELECT  u.user_id, u.name, m.seller_score, m.avg_rating, m.review_count,
        m.orders_total,
        CASE WHEN m.orders_total = 0 THEN 0
             ELSE round(100.0 * m.orders_returned / m.orders_total, 1) END AS return_rate_pct,
        m.avg_response_mins, m.is_flagged
FROM seller_metrics m JOIN users u ON u.user_id = m.seller_id
ORDER BY m.seller_score DESC;

-- Materialised view: demand per category per ~1km grid cell (dynamic pricing hints)
CREATE MATERIALIZED VIEW mv_demand_by_cell AS
SELECT  l.category_id,
        ST_SnapToGrid(l.location::geometry, 0.01) AS cell,          -- ~1.1 km grid
        count(*) FILTER (WHERE l.status = 'active')  AS active_supply,
        count(*) FILTER (WHERE l.status = 'sold')    AS sold_count,
        COALESCE(sum(m.buyers), 0)                   AS interested_buyers,
        round(avg(l.price), 2)                       AS avg_price
FROM listings l
-- pre-aggregate messages per listing so the join does not multiply listing rows
LEFT JOIN (SELECT listing_id, count(DISTINCT sender_id) AS buyers
           FROM messages GROUP BY listing_id) m ON m.listing_id = l.listing_id
GROUP BY l.category_id, ST_SnapToGrid(l.location::geometry, 0.01);
CREATE INDEX idx_mv_demand ON mv_demand_by_cell (category_id);
```

SQL DML Statements

a) Queries and DML

File: db/postgres/queries/queries.sql (run after 03_seed.sql, which inserts 10 users, the categories, zones, 19 listings, messages, orders and reviews). It has the proximity search, the EXPLAIN check of the spatial index, joins, GROUP BY and HAVING, a correlated subquery, the full text search, and INSERT, UPDATE and DELETE examples including the statements that must fail.

```
-- Vicinia :: important queries (DML + analytics).  Run after 03_seed.sql

-- Q1. Proximity search: couches within 5 km of Indiranagar, price<=15000, rating>=0,
--     ranked by blended seller score + distance.  Uses GiST index via ST_DWithin.
SELECT * FROM search_nearby(12.9784, 77.6408, 5000, NULL, NULL, 15000, NULL, 'couch', 'rank', 20);

-- Q1b. Same, sorted purely by distance, using the KNN operator (index-ordered nearest-first)
SELECT l.listing_id, l.title, l.price,
       round((l.location <-> ST_SetSRID(ST_MakePoint(77.6408, 12.9784), 4326)::geography)::NUMERIC) AS
dist_m
FROM listings l
WHERE l.status = 'active'
ORDER BY l.location <-> ST_SetSRID(ST_MakePoint(77.6408, 12.9784), 4326)::geography
LIMIT 5;

-- Q2. EXPLAIN proof that the spatial index is used
EXPLAIN (ANALYZE, BUFFERS)
SELECT listing_id FROM listings
WHERE ST_DWithin(location, ST_SetSRID(ST_MakePoint(77.6408, 12.9784), 4326)::geography, 5000);

-- Q3. Purchase (race-safe).  Returns new order id; second attempt fails with "no longer available"
-- BEGIN; SELECT purchase_listing(2, 1); COMMIT;

-- Q4. Seller leaderboard with return rate (JOIN + aggregate via view)
SELECT * FROM v_seller_leaderboard;

-- Q5. Sellers whose average rating is high but who have fewer than 3 distinct buyers
--     (GROUP BY / HAVING + COUNT DISTINCT) -- a simple red flag
SELECT o.seller_id, u.name, round(avg(r.rating), 2) AS avg_rating,
       count(DISTINCT o.buyer_id) AS distinct_buyers, count(*) AS orders
FROM orders o
JOIN users u        ON u.user_id = o.seller_id
JOIN reviews r      ON r.order_id = o.order_id
GROUP BY o.seller_id, u.name
HAVING avg(r.rating) >= 4.5 AND count(DISTINCT o.buyer_id) < 3
ORDER BY avg_rating DESC;

-- Q6. Subquery: listings priced below the average of their category
SELECT l.listing_id, l.title, l.price, c.name AS category
FROM listings l JOIN categories c ON c.category_id = l.category_id
WHERE l.status = 'active'
  AND l.price < (SELECT avg(price) FROM listings x WHERE x.category_id = l.category_id AND x.status =
'active')
ORDER BY l.price;

-- Q7. Revenue per seller per month (aggregate + date_trunc)
SELECT s.name AS seller, date_trunc('month', o.placed_at) AS month,
       count(*) AS orders, sum(o.total_amount) AS revenue
FROM orders o JOIN users s ON s.user_id = o.seller_id
WHERE o.status IN ('placed', 'delivered')
GROUP BY s.name, date_trunc('month', o.placed_at)
ORDER BY month DESC, revenue DESC;

-- Q8. "Bought this, also bought that" (SQL co-purchase; Neo4j gives the graph version)
SELECT l2.listing_id, l2.title, count(*) AS co_buyers
FROM order_items a
JOIN orders oa ON oa.order_id = a.order_id
JOIN orders ob ON ob.buyer_id = oa.buyer_id AND ob.order_id <> oa.order_id
JOIN order_items b ON b.order_id = ob.order_id
JOIN listings l1 ON l1.listing_id = a.listing_id
JOIN listings l2 ON l2.listing_id = b.listing_id AND l2.category_id = l1.category_id
WHERE a.listing_id = 10
GROUP BY l2.listing_id, l2.title
```

```
ORDER BY co_buyers DESC LIMIT 5;

-- Q9. Dynamic pricing hint + delivery estimate
SELECT * FROM pricing_hint(1);
SELECT * FROM estimate_delivery(1, ST_SetSRID(ST_MakePoint(77.7500, 12.9698), 4326)::geography);

-- Q10. Full-text search with ranking
SELECT listing_id, title, ts_rank(search_vec, q) AS rank
FROM listings, plainto_tsquery('english', 'leather sofa') q
WHERE search_vec @@ q ORDER BY rank DESC;

-- DML demonstrations
-- INSERT
INSERT INTO listings (seller_id, category_id, title, description, price, location)
VALUES (5, 1, 'Wooden bookshelf', '5-shelf, sturdy', 2500, ST_SetSRID(ST_MakePoint(77.6390, 12.9115),
4326)::geography);

-- UPDATE (fires price_history trigger)
UPDATE listings SET price = 2300 WHERE title = 'Wooden bookshelf';
SELECT * FROM price_history ORDER BY changed_at DESC LIMIT 3;

-- UPDATE: mark order returned (fires score trigger)
-- CALL set_order_status(3, 'returned');

-- DELETE (cascades to price_history)
DELETE FROM listings WHERE title = 'Wooden bookshelf';

-- Integrity demos (each should FAIL):
-- INSERT INTO listings (seller_id, category_id, title, price, location) VALUES (5,1,'x',-5,
ST_GeogFromText('POINT(77 12)'));   -- price check + title length
-- INSERT INTO reviews (order_id, reviewer_id, seller_id, rating) VALUES (3, 4, 7, 5);
-- order not delivered
-- INSERT INTO order_items VALUES (1, 10, 300);
-- listing already sold (UNIQUE)

-- CONCURRENCY DEMO (run in two psql sessions)
-- Session A:  BEGIN; SELECT purchase_listing(2, 1);        -- holds row lock, do NOT commit yet
-- Session B:  BEGIN; SELECT purchase_listing(3, 1);        -- BLOCKS waiting on A's lock
-- Session A:  COMMIT;                                       -- A wins
-- Session B:  -> ERROR: Listing 1 is no longer available (status: sold)
```

b) Functions, Procedure and Triggers

File: db/postgres/02_functions.sql. purchase_listing locks the listing row with SELECT FOR UPDATE, so the second buyer waits and then sees the item as sold. search_nearby ranks by 0.6 x (S / 100) + 0.4 x (1 - d / D), where S is the seller score, d the distance and D the radius. The seller score S = 0.40 R + 0.25 P + 0.20 V + 0.15 T (rating, returns, reply speed, network trust) is multiplied by 0.25 for a flagged seller. Ratings are pulled toward 3 with (avg x n + 15) / (n + 5) so that one review cannot give a perfect score.

```
-- Vicinia :: Functions, stored procedures, triggers

-- 1. Seller score
--    score = 40% rating + 25% (1 - return rate) + 20% responsiveness + 15% graph trust
--    Result is 0..100 (x0.25 if flagged as part of a collusion ring).
--    Called by triggers; feeds search ranking.
CREATE OR REPLACE FUNCTION compute_seller_score(p_seller BIGINT)
RETURNS NUMERIC AS $$
DECLARE
    m seller_metrics%ROWTYPE;
    rating_part  NUMERIC;
    return_part  NUMERIC;
    resp_part    NUMERIC;
    trust_part   NUMERIC;
    final_score  NUMERIC;
BEGIN
    SELECT * INTO m FROM seller_metrics WHERE seller_id = p_seller;
    IF NOT FOUND THEN RETURN 50; END IF;

    -- rating: 0..5 -> 0..100; shrink toward neutral (3.0) when there are few reviews
    rating_part := ((m.avg_rating * m.review_count + 3.0 * 5) / (m.review_count + 5)) / 5 * 100;
```

```
    -- return rate: 0% returns -> 100, 50%+ returns -> 0
    return_part := 100 * GREATEST(0, 1 - 2 * CASE WHEN m.orders_total = 0 THEN 0
                                                  ELSE m.orders_returned::NUMERIC / m.orders_total END);
    -- response time: <=10 min -> 100, >=24h -> 0 (linear), no data -> neutral 50
    resp_part := CASE WHEN m.avg_response_mins = 0 THEN 50
                      ELSE 100 * GREATEST(0, LEAST(1, (1440 - m.avg_response_mins) / (1440 - 10))) END;
    trust_part := LEAST(100, GREATEST(0, m.trust_boost));

    final_score := round(0.40 * rating_part + 0.25 * return_part + 0.20 * resp_part + 0.15 * trust_part,
2);
    -- sellers flagged by graph collusion detection are heavily demoted in search ranking
    IF m.is_flagged THEN final_score := round(final_score * 0.25, 2); END IF;
    RETURN final_score;
END;
$$ LANGUAGE plpgsql STABLE;

-- Recalculate one seller's metrics from raw tables and store the new score
CREATE OR REPLACE FUNCTION refresh_seller_metrics(p_seller BIGINT)
RETURNS VOID AS $$
BEGIN
    INSERT INTO seller_metrics (seller_id) VALUES (p_seller) ON CONFLICT DO NOTHING;

    UPDATE seller_metrics sm SET
        avg_rating      = COALESCE((SELECT round(avg(rating), 2) FROM reviews WHERE seller_id = p_seller),
0),
        review_count    = (SELECT count(*) FROM reviews WHERE seller_id = p_seller),
        orders_total    = (SELECT count(*) FROM orders WHERE seller_id = p_seller AND status <>
'cancelled'),
        orders_returned = (SELECT count(*) FROM orders WHERE seller_id = p_seller AND status = 'returned'),
        -- response time: gap between a buyer's first message on a listing and the seller's first reply
        avg_response_mins = COALESCE((
            SELECT round(avg(EXTRACT(EPOCH FROM (reply.t - q.t)) / 60)::NUMERIC, 2)
            FROM (SELECT msg.listing_id, msg.sender_id, min(msg.sent_at) AS t
                  FROM messages msg JOIN listings l ON l.listing_id = msg.listing_id
                  WHERE l.seller_id = p_seller AND msg.sender_id <> p_seller
                  GROUP BY msg.listing_id, msg.sender_id) q
            JOIN LATERAL (SELECT min(r.sent_at) AS t FROM messages r
                          WHERE r.listing_id = q.listing_id AND r.sender_id = p_seller
                            AND r.receiver_id = q.sender_id AND r.sent_at > q.t) reply ON reply.t IS NOT
NULL
        ), 0),
        updated_at = now()
    WHERE sm.seller_id = p_seller;

    UPDATE seller_metrics SET seller_score = compute_seller_score(p_seller)
    WHERE seller_id = p_seller;
END;
$$ LANGUAGE plpgsql;

-- 2. Triggers

-- 2a. Create a metrics row automatically for every new seller
CREATE OR REPLACE FUNCTION trg_user_seller_metrics() RETURNS TRIGGER AS $$
BEGIN
    IF NEW.role = 'seller' THEN
        INSERT INTO seller_metrics (seller_id) VALUES (NEW.user_id) ON CONFLICT DO NOTHING;
    END IF;
    RETURN NEW;
END; $$ LANGUAGE plpgsql;

CREATE TRIGGER user_seller_metrics
AFTER INSERT OR UPDATE OF role ON users
FOR EACH ROW EXECUTE FUNCTION trg_user_seller_metrics();

-- 2b. New / changed review -> recompute seller score
CREATE OR REPLACE FUNCTION trg_review_score() RETURNS TRIGGER AS $$
BEGIN
    PERFORM refresh_seller_metrics(COALESCE(NEW.seller_id, OLD.seller_id));
    RETURN NULL;
END; $$ LANGUAGE plpgsql;

CREATE TRIGGER review_changes_score
AFTER INSERT OR UPDATE OR DELETE ON reviews
FOR EACH ROW EXECUTE FUNCTION trg_review_score();

-- 2c. Order inserted / status changed (e.g. returned) -> recompute seller score
CREATE OR REPLACE FUNCTION trg_order_score() RETURNS TRIGGER AS $$
```

```
BEGIN
    PERFORM refresh_seller_metrics(NEW.seller_id);
    RETURN NULL;
END; $$ LANGUAGE plpgsql;

CREATE TRIGGER order_changes_score
AFTER INSERT OR UPDATE OF status ON orders
FOR EACH ROW EXECUTE FUNCTION trg_order_score();

-- 2d. Log price changes (feeds dynamic pricing)
CREATE OR REPLACE FUNCTION trg_price_history() RETURNS TRIGGER AS $$
BEGIN
    IF NEW.price IS DISTINCT FROM OLD.price THEN
        INSERT INTO price_history (listing_id, old_price, new_price)
        VALUES (OLD.listing_id, OLD.price, NEW.price);
    END IF;
    RETURN NEW;
END; $$ LANGUAGE plpgsql;

CREATE TRIGGER listing_price_history
AFTER UPDATE OF price ON listings
FOR EACH ROW EXECUTE FUNCTION trg_price_history();

-- 2e. Integrity: only the buyer of a DELIVERED order can review it, and the seller must match
CREATE OR REPLACE FUNCTION trg_review_integrity() RETURNS TRIGGER AS $$
DECLARE o orders%ROWTYPE;
BEGIN
    SELECT * INTO o FROM orders WHERE order_id = NEW.order_id;
    IF o.buyer_id <> NEW.reviewer_id THEN
        RAISE EXCEPTION 'Only the buyer of order % may review it', NEW.order_id;
    END IF;
    IF o.seller_id <> NEW.seller_id THEN
        RAISE EXCEPTION 'Review seller does not match order seller';
    END IF;
    IF o.status <> 'delivered' THEN
        RAISE EXCEPTION 'Order % is not delivered yet (status: %)', NEW.order_id, o.status;
    END IF;
    RETURN NEW;
END; $$ LANGUAGE plpgsql;

CREATE TRIGGER review_integrity
BEFORE INSERT ON reviews
FOR EACH ROW EXECUTE FUNCTION trg_review_integrity();

-- 2f. A seller's listing must not be edited once sold
CREATE OR REPLACE FUNCTION trg_block_sold_edit() RETURNS TRIGGER AS $$
BEGIN
    IF OLD.status = 'sold' AND NEW.status <> 'sold' THEN
        RAISE EXCEPTION 'Listing % is already sold and cannot be reopened', OLD.listing_id;
    END IF;
    RETURN NEW;
END; $$ LANGUAGE plpgsql;

CREATE TRIGGER listing_block_sold_edit
BEFORE UPDATE ON listings
FOR EACH ROW EXECUTE FUNCTION trg_block_sold_edit();

-- 3. Delivery-time estimate
--    hours = zone.base_hours + zone.per_km_hours * distance_km(listing -> buyer)
--    Zone is the polygon that contains the buyer's location (falls back to nearest zone).
CREATE OR REPLACE FUNCTION estimate_delivery(p_listing BIGINT, p_buyer_location GEOGRAPHY)
RETURNS TABLE (zone_id INT, zone_name TEXT, distance_km NUMERIC, est_hours NUMERIC) AS $$
    SELECT z.zone_id, z.name::TEXT,
           round((ST_Distance(l.location, p_buyer_location) / 1000)::NUMERIC, 2),
           round(z.base_hours + z.per_km_hours * (ST_Distance(l.location, p_buyer_location) /
1000)::NUMERIC, 1)
    FROM listings l
    CROSS JOIN LATERAL (
        SELECT * FROM delivery_zones dz
        ORDER BY (NOT ST_Covers(dz.boundary, p_buyer_location)), dz.boundary <-> p_buyer_location
        LIMIT 1
    ) z
    WHERE l.listing_id = p_listing;
$$ LANGUAGE sql STABLE;

-- 4. Race-condition-safe purchase
--
```

```
--    Two buyers click "Buy" on the same couch at the same moment.
--    SELECT ... FOR UPDATE takes a row lock on the listing: the second
--    transaction BLOCKS until the first commits, then re-reads status='sold'
--    and fails cleanly.  The UNIQUE(listing_id) on order_items is a second
--    safety net at the schema level.
--
--    Caller should run this inside a transaction with
--    ISOLATION LEVEL READ COMMITTED (default; sufficient because the row
--    lock serialises the contested row) -- see docs for the REPEATABLE READ variant.
CREATE OR REPLACE FUNCTION purchase_listing(p_buyer BIGINT, p_listing BIGINT)
RETURNS BIGINT AS $$
DECLARE
    l        listings%ROWTYPE;
    buyer    users%ROWTYPE;
    v_zone   INT;
    v_hours  NUMERIC;
    v_order  BIGINT;
BEGIN
    SELECT * INTO buyer FROM users WHERE user_id = p_buyer AND role IN ('buyer','seller');
    IF NOT FOUND THEN RAISE EXCEPTION 'Unknown buyer %', p_buyer; END IF;

    -- Row-level lock: concurrent purchasers queue here
    SELECT * INTO l FROM listings WHERE listing_id = p_listing FOR UPDATE;
    IF NOT FOUND THEN RAISE EXCEPTION 'Listing % does not exist', p_listing; END IF;
    IF l.status <> 'active' THEN
        RAISE EXCEPTION 'Listing % is no longer available (status: %)', p_listing, l.status
            USING ERRCODE = 'P0001';
    END IF;
    IF l.seller_id = p_buyer THEN
        RAISE EXCEPTION 'You cannot buy your own listing';
    END IF;

    SELECT e.zone_id, e.est_hours INTO v_zone, v_hours FROM estimate_delivery(p_listing, buyer.location) e;

    INSERT INTO orders (buyer_id, seller_id, total_amount, zone_id, est_hours)
    VALUES (p_buyer, l.seller_id, l.price, v_zone, v_hours)
    RETURNING order_id INTO v_order;

    INSERT INTO order_items (order_id, listing_id, unit_price) VALUES (v_order, p_listing, l.price);
    UPDATE listings SET status = 'sold' WHERE listing_id = p_listing;

    RETURN v_order;
END;
$$ LANGUAGE plpgsql;

-- Mark an order delivered / returned (re-opens nothing; just state change + score refresh via trigger)
CREATE OR REPLACE PROCEDURE set_order_status(p_order BIGINT, p_status order_status)
LANGUAGE plpgsql AS $$
BEGIN
    UPDATE orders
       SET status = p_status,
           delivered_at = CASE WHEN p_status = 'delivered' THEN now() ELSE delivered_at END
     WHERE order_id = p_order;
    IF NOT FOUND THEN RAISE EXCEPTION 'Order % not found', p_order; END IF;
END; $$;

-- 5. Proximity search (spatial index + dynamic ranking)
--    rank = 0.6 * seller_score/100 + 0.4 * (1 - distance/radius)
--    Uses ST_DWithin (index-assisted filter) then sorts by blended rank.
CREATE OR REPLACE FUNCTION search_nearby(
    p_lat DOUBLE PRECISION, p_lon DOUBLE PRECISION, p_radius_m DOUBLE PRECISION DEFAULT 5000,
    p_category INT DEFAULT NULL, p_min_price NUMERIC DEFAULT NULL, p_max_price NUMERIC DEFAULT NULL,
    p_min_rating NUMERIC DEFAULT NULL, p_query TEXT DEFAULT NULL,
    p_sort TEXT DEFAULT 'rank', p_limit INT DEFAULT 50)
RETURNS TABLE (listing_id BIGINT, title TEXT, price NUMERIC, category TEXT, condition TEXT,
               seller_id BIGINT, seller_name TEXT, seller_score NUMERIC, avg_rating NUMERIC,
               seller_flagged BOOLEAN, distance_m DOUBLE PRECISION, rank_score DOUBLE PRECISION,
               lat DOUBLE PRECISION, lon DOUBLE PRECISION) AS $$
    WITH origin AS (SELECT ST_SetSRID(ST_MakePoint(p_lon, p_lat), 4326)::geography AS g)
    SELECT v.listing_id, v.title::TEXT, v.price, v.category::TEXT, v.condition::TEXT,
           v.seller_id, v.seller_name::TEXT, v.seller_score, v.avg_rating, v.seller_flagged,
           ST_Distance(v.location, o.g) AS distance_m,
           (0.6 * v.seller_score / 100.0 + 0.4 * (1 - ST_Distance(v.location, o.g) / p_radius_m))::DOUBLE
PRECISION AS rank_score,
           ST_Y(v.location::geometry), ST_X(v.location::geometry)
    FROM v_listing_search v, origin o
    WHERE v.status = 'active'
```

```
      AND ST_DWithin(v.location, o.g, p_radius_m)
      AND (p_category  IS NULL OR v.category_id = p_category)
      AND (p_min_price IS NULL OR v.price >= p_min_price)
      AND (p_max_price IS NULL OR v.price <= p_max_price)
      AND (p_min_rating IS NULL OR v.avg_rating >= p_min_rating)
      AND (p_query IS NULL OR v.search_vec @@ plainto_tsquery('english', p_query))
    ORDER BY CASE p_sort WHEN 'price_asc'  THEN v.price END ASC,
             CASE p_sort WHEN 'price_desc' THEN v.price END DESC,
             CASE p_sort WHEN 'distance'   THEN ST_Distance(v.location, o.g) END ASC,
             (0.6 * v.seller_score / 100.0 + 0.4 * (1 - ST_Distance(v.location, o.g) / p_radius_m)) DESC
    LIMIT p_limit;
$$ LANGUAGE sql STABLE;

-- 6. Dynamic pricing hint
--    Compares demand (distinct interested buyers) vs supply in ~1km cells around the listing.
--    ratio > 2 -> suggest +10%; ratio < 0.5 -> suggest -10%; otherwise hold.
CREATE OR REPLACE FUNCTION pricing_hint(p_listing BIGINT)
RETURNS TABLE (current_price NUMERIC, area_avg_price NUMERIC, demand INT, supply INT,
               demand_supply_ratio NUMERIC, suggested_price NUMERIC, hint TEXT) AS $$
    WITH me AS (SELECT * FROM listings WHERE listing_id = p_listing),
    area AS (
        SELECT COALESCE(sum(c.interested_buyers), 0)::INT AS demand,
               COALESCE(sum(c.active_supply), 0)::INT     AS supply,
               avg(c.avg_price)                           AS avg_price
        FROM me
        JOIN mv_demand_by_cell c
          ON c.category_id = me.category_id
         AND ST_DWithin(c.cell::geography, me.location, 2000)
    )
    SELECT me.price, round(a.avg_price, 2), a.demand, a.supply,
           round(a.demand::NUMERIC / GREATEST(a.supply, 1), 2),
           CASE WHEN a.demand::NUMERIC / GREATEST(a.supply, 1) > 2   THEN round(me.price * 1.10, 2)
                WHEN a.demand::NUMERIC / GREATEST(a.supply, 1) < 0.5 THEN round(me.price * 0.90, 2)
                ELSE me.price END,
           CASE WHEN a.demand::NUMERIC / GREATEST(a.supply, 1) > 2   THEN 'High demand nearby: consider
raising the price ~10%'
                WHEN a.demand::NUMERIC / GREATEST(a.supply, 1) < 0.5 THEN 'Low demand nearby: a ~10% cut
may help it sell'
                ELSE 'Price is in line with local demand' END
    FROM me, area a;
$$ LANGUAGE sql STABLE;

-- Sync endpoint: Neo4j pushes graph trust (0..100) and collusion flag back into Postgres
CREATE OR REPLACE FUNCTION apply_graph_signals(p_seller BIGINT, p_trust NUMERIC, p_flagged BOOLEAN)
RETURNS VOID AS $$
BEGIN
    INSERT INTO seller_metrics (seller_id) VALUES (p_seller) ON CONFLICT DO NOTHING;
    UPDATE seller_metrics
       SET trust_boost = LEAST(100, GREATEST(0, p_trust)), is_flagged = p_flagged
     WHERE seller_id = p_seller;
    PERFORM refresh_seller_metrics(p_seller);
END; $$ LANGUAGE plpgsql;
```

NoSQL/ Vector Database Commands

Neo4j graph. A User node has the same id as users.user_id in PostgreSQL. There are three relationship types: TRUSTS (weight 1 to 5), BOUGHT_FROM (one per order, buyer to seller) and REVIEWED (rating). The seed has 10 users and a planted ring of users 8, 9 and 10 who only trust each other and only buy from each other.



Figure 5: Graph model

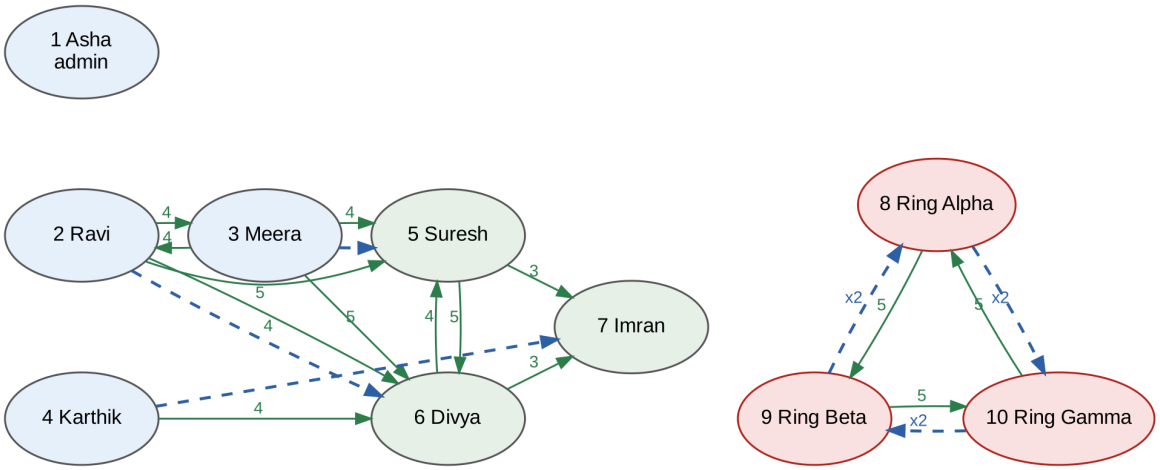


Figure 6: Seed graph (the collusion ring is in red)

a) Constraints, indexes and seed data

```
// Vicinia :: Neo4j graph model (trust & collusion detection)
//
// Nodes
//   (:User {id, name, role, trustScore, flagged})        id mirrors users.user_id in PostgreSQL
// Relationships
//   (:User)-[:TRUSTS {weight, since}]->(:User)           explicit trust (1..5)
//   (:User)-[:BOUGHT_FROM {orderId, amount, at}]->(:User) one edge per completed order
//   (:User)-[:REVIEWED {orderId, rating, at}]->(:User)    reviewer -> seller

// ---- Constraints & indexes ----
CREATE CONSTRAINT user_id_unique IF NOT EXISTS FOR (u:User) REQUIRE u.id IS UNIQUE;
CREATE INDEX user_flagged IF NOT EXISTS FOR (u:User) ON (u.flagged);
CREATE INDEX bought_order IF NOT EXISTS FOR ()-[b:BOUGHT_FROM]-() ON (b.orderId);

// ---- Seed (mirrors Postgres seed) ----
UNWIND [
  {id:1,name:'Asha Admin',role:'admin'},   {id:2,name:'Ravi Buyer',role:'buyer'},
  {id:3,name:'Meera Buyer',role:'buyer'},  {id:4,name:'Karthik Buyer',role:'buyer'},
  {id:5,name:'Suresh Seller',role:'seller'},{id:6,name:'Divya Seller',role:'seller'},
  {id:7,name:'Imran Seller',role:'seller'}, {id:8,name:'Ring Alpha',role:'seller'},
```

```
  {id:9,name:'Ring Beta',role:'seller'},    {id:10,name:'Ring Gamma',role:'seller'}
] AS row
MERGE (u:User {id: row.id}) SET u.name = row.name, u.role = row.role, u.trustScore = 0, u.flagged = false;

// Explicit trust edges (community members vouching for each other)
UNWIND [
  [2,5,5],[2,6,4],[3,6,5],[3,5,4],[4,6,4],[5,6,5],[6,5,4],[6,7,3],[5,7,3],[3,2,4],[2,3,4],
  [8,9,5],[9,10,5],[10,8,5]               // ring members only trust each other
] AS t
MATCH (a:User {id:t[0]}), (b:User {id:t[1]})
MERGE (a)-[r:TRUSTS]->(b) SET r.weight = t[2], r.since = date();

// Purchase + review edges (same orders as Postgres seed)
UNWIND [
  {o:1,buyer:2,seller:6,amt:9000,rating:5},  {o:2,buyer:3,seller:5,amt:9000,rating:4},
  {o:3,buyer:4,seller:7,amt:11000,rating:null},
  {o:4,buyer:9,seller:8,amt:300,rating:5},   {o:5,buyer:9,seller:8,amt:250,rating:5},
  {o:6,buyer:10,seller:9,amt:200,rating:5},  {o:7,buyer:10,seller:9,amt:350,rating:5},
  {o:8,buyer:8,seller:10,amt:150,rating:5},  {o:9,buyer:8,seller:10,amt:600,rating:5}
] AS x
MATCH (b:User {id:x.buyer}), (s:User {id:x.seller})
MERGE (b)-[bf:BOUGHT_FROM {orderId:x.o}]->(s) SET bf.amount = x.amt, bf.at = datetime()
FOREACH (_ IN CASE WHEN x.rating IS NULL THEN [] ELSE [1] END |
  MERGE (b)-[rv:REVIEWED {orderId:x.o}]->(s) SET rv.rating = x.rating, rv.at = datetime());
```

b) Queries: trust, collusion detection, recommendations

```
// Vicinia :: Cypher queries used by the app (parameters prefixed with $)

// ---- 1. Personalised trust: how much does $viewer trust $seller through the network? ----
// Paths up to 3 hops; each hop multiplies weight/5 and halves per extra hop (decay).
MATCH p = (v:User {id: $viewer})-[:TRUSTS*1..3]->(s:User {id: $seller})
WHERE all(n IN nodes(p) WHERE coalesce(n.flagged, false) = false)
WITH p, reduce(w = 1.0, r IN relationships(p) | w * r.weight / 5.0) * (0.5 ^ (length(p) - 1)) AS pathTrust
RETURN round(100 * (1 - exp(-sum(pathTrust))), 1) AS trust, count(p) AS paths;   // 0..100, saturating

// ---- 2. Global trust boost per seller (pushed to PostgreSQL seller_metrics.trust_boost) ----
// Each incoming TRUSTS counts more if the truster is itself trusted (has incoming trust); flagged users
carry no weight.
MATCH (t:User)-[r:TRUSTS]->(s:User {role: 'seller'})
WHERE coalesce(t.flagged, false) = false
WITH s, sum(r.weight / 5.0 * (1 + size([(x:User)-[:TRUSTS]->(t) WHERE coalesce(x.flagged, false) = false |
x]))) AS raw
SET s.trustScore = round(100 * raw / (raw + 2), 1)
RETURN s.id AS sellerId, s.trustScore AS trust;

// ---- 3. Collusion-ring detection ----
// A ring = a closed buying cycle among sellers (A->B->C->A, length 2..4) whose members
// receive (almost) NO purchases from users outside the cycle.
MATCH p = (a:User)-[:BOUGHT_FROM*2..4]->(a)
WHERE all(n IN nodes(p)[0..-1] WHERE n.role = 'seller')
WITH DISTINCT [n IN nodes(p)[0..-1] | n] AS members
WITH members, apoc.coll.sort([m IN members | m.id]) AS key
WITH DISTINCT key AS ids, members
UNWIND members AS m
OPTIONAL MATCH (x:User)-[:BOUGHT_FROM]->(m) WHERE NOT x IN members
WITH ids, members, count(DISTINCT x) AS outsideBuyers
WHERE outsideBuyers = 0
RETURN ids AS ringMembers, size(members) AS size, outsideBuyers;

// Plain-Cypher variant of the 3-cycle check (no APOC needed):
MATCH (a:User)-[:BOUGHT_FROM]->(b:User)-[:BOUGHT_FROM]->(c:User)-[:BOUGHT_FROM]->(a)
WHERE a.id < b.id AND a.id < c.id
WITH [a, b, c] AS ring
WHERE NOT EXISTS {
  MATCH (x:User)-[:BOUGHT_FROM]->(m:User) WHERE m IN ring AND NOT x IN ring
}
RETURN [m IN ring | m.id] AS ringMembers;

// ---- 4. Flag / unflag users in a detected ring ----
MATCH (a:User)-[:BOUGHT_FROM]->(b:User)-[:BOUGHT_FROM]->(c:User)-[:BOUGHT_FROM]->(a)
WHERE a.id < b.id AND a.id < c.id
WITH [a, b, c] AS ring
WHERE NOT EXISTS { MATCH (x:User)-[:BOUGHT_FROM]->(m:User) WHERE m IN ring AND NOT x IN ring }
UNWIND ring AS m
SET m.flagged = true
RETURN collect(DISTINCT m.id) AS flaggedIds;
```

```
// ---- 5. Suspicious reciprocal 5-star reviews (A reviews B 5 and B reviews A 5) ----
MATCH (a:User)-[r1:REVIEWED]->(b:User)-[r2:REVIEWED]->(a)
WHERE a.id < b.id AND r1.rating >= 4 AND r2.rating >= 4
RETURN a.id AS a, b.id AS b, count(*) AS pairs;

// ---- 6. "People like you also bought from" (seller recommendations) ----
MATCH (me:User {id:
$userId})-[:BOUGHT_FROM]->(s:User)<-[:BOUGHT_FROM]-(other:User)-[:BOUGHT_FROM]->(rec:User)
WHERE rec <> me AND NOT (me)-[:BOUGHT_FROM]->(rec) AND coalesce(rec.flagged, false) = false
RETURN rec.id AS sellerId, rec.name AS name, count(DISTINCT other) AS supporters
ORDER BY supporters DESC LIMIT 5;

// ---- 7. Sync on order completion (called from the API after Postgres commit) ----
MERGE (b:User {id: $buyerId}) MERGE (s:User {id: $sellerId})
MERGE (b)-[r:BOUGHT_FROM {orderId: $orderId}]->(s) SET r.amount = $amount, r.at = datetime();
```

Results

a) Resulting Tables' Screenshots

Test results (SQL tests, concurrency test, EXPLAIN ANALYZE and API checks):

Test	What it checks	Result
T1	All results are inside the search radius	Pass
T2	Sold items never appear in search	Pass
T3	Purchase works, a second purchase is rejected	Pass
T4	Cannot buy your own listing	Pass
T5	Review on an undelivered order is rejected	Pass
T6	A 1 star review lowers the score	Pass (70.33 to 67.86)
T7	A return lowers the score	Pass (71.53 to 46.53)
T8	A flagged seller drops	Pass (63.57 to 15.89)
T9	Bad price, rating or email rejected by the database	Pass
T10, T11	Price change is logged; delivery estimate	Pass
Concurrency	Two buyers, one item, at READ COMMITTED, REPEATABLE READ and SERIALIZABLE	Exactly one winner each time
EXPLAIN ANALYZE	20,000 listings, radius search	Bitmap index scan on idx_listings_location, about 7 ms
API	Anonymous request to a protected route; wrong password	401
API	Short password; registering as admin	400
API	Buyer opens an admin route; buyer creates a listing	403
Graph	Collusion scan flags users 8, 9, 10; trusting a seller raises network trust	To be run with Neo4j

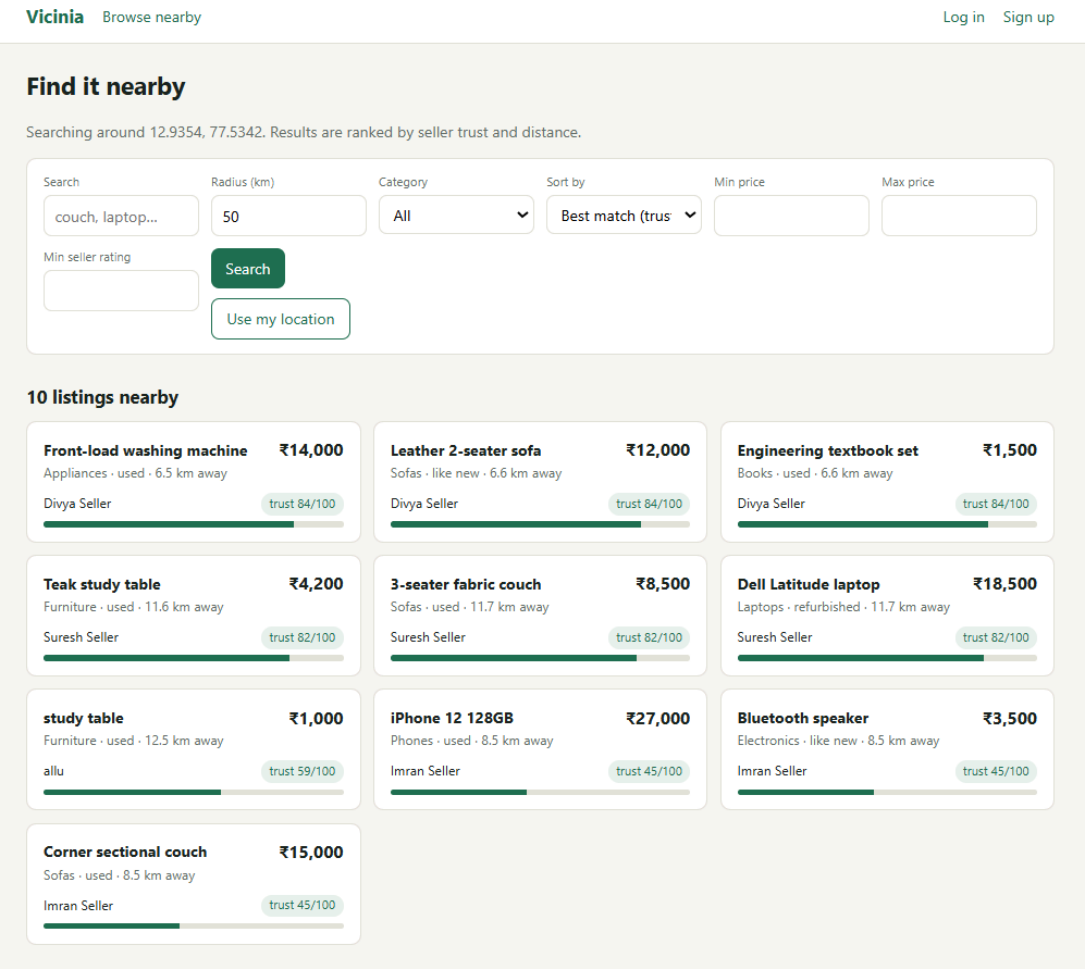
Next.js-Project Tree Structure

```
vicinia/
├── app/
│   ├── admin/
│   │   └── page.tsx
│   ├── api/
│   │   ├── admin/
│   │   │   ├── detect/
│   │   │   │   └── route.ts
│   │   │   ├── moderate/
│   │   │   │   └── route.ts
│   │   │   └── overview/
│   │   │       └── route.ts
│   │   ├── auth/
│   │   │   ├── login/
│   │   │   │   └── route.ts
│   │   │   ├── logout/
│   │   │   │   └── route.ts
│   │   │   ├── me/
│   │   │   │   └── route.ts
│   │   │   └── register/
│   │   │       └── route.ts
│   │   ├── categories/
│   │   │   └── route.ts
│   │   ├── listings/
│   │   │   ├── [id]/
│   │   │   │   └── route.ts
│   │   │   └── route.ts
│   │   ├── orders/
│   │   │   ├── [id]/
│   │   │   │   └── route.ts
│   │   │   └── route.ts
│   │   ├── recommendations/
│   │   │   └── route.ts
│   │   ├── reviews/
│   │   │   └── route.ts
│   │   ├── seller/
│   │   │   └── dashboard/
│   │   │       └── route.ts
│   │   └── trust/
│   │       └── route.ts
│   ├── listing/
│   │   └── [id]/
│   │       └── page.tsx
│   ├── login/
│   │   └── page.tsx
│   ├── orders/
│   │   └── page.tsx
│   ├── register/
│   │   └── page.tsx
│   ├── sell/
│   │   └── page.tsx
│   ├── globals.css
│   ├── layout.tsx
│   └── page.tsx
├── components/
│   └── Nav.tsx
├── db/
│   ├── neo4j/
│   │   ├── 01_schema_seed.cypher
│   │   └── 02_queries.cypher
│   └── postgres/
│       ├── queries/
```

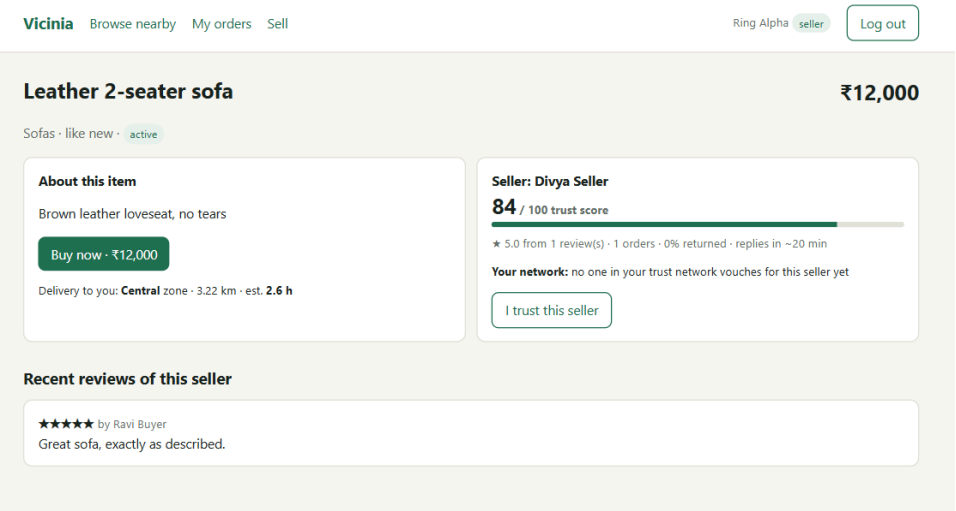
```
│       │   └── queries.sql
│       ├── 01_schema.sql
│       ├── 02_functions.sql
│       └── 03_seed.sql
├── docs/
│   └── PROJECT_DOCUMENTATION.md
├── lib/
│   ├── auth.ts
│   ├── client.ts
│   ├── graph.ts
│   ├── http.ts
│   ├── neo.ts
│   └── pg.ts
├── scripts/
│   └── seed-neo4j.mjs
├── tests/
│   ├── concurrency.mjs
│   └── sql_tests.sql
├── .env.example
├── .gitignore
├── docker-compose.yml
├── next.config.mjs
├── package.json
├── README.md
└── tsconfig.json
```

Few sample Screenshots of application code and demo

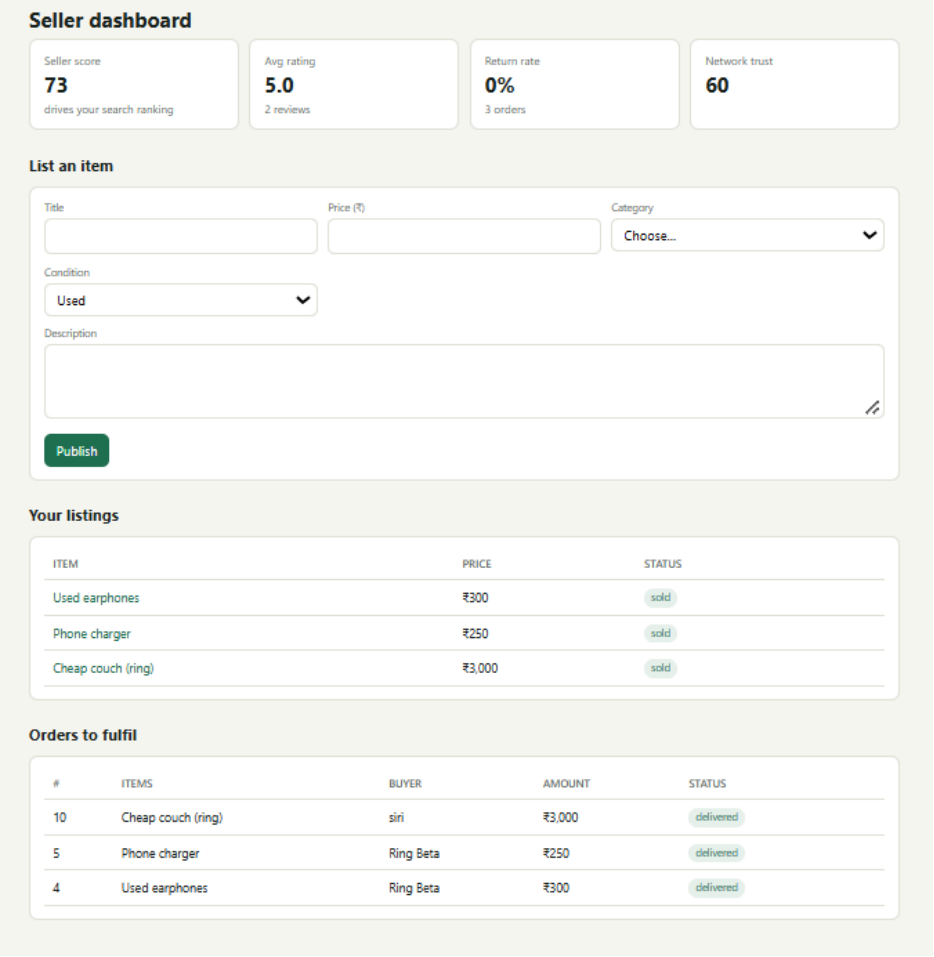
[Screenshot: search page with filters and ranked results]



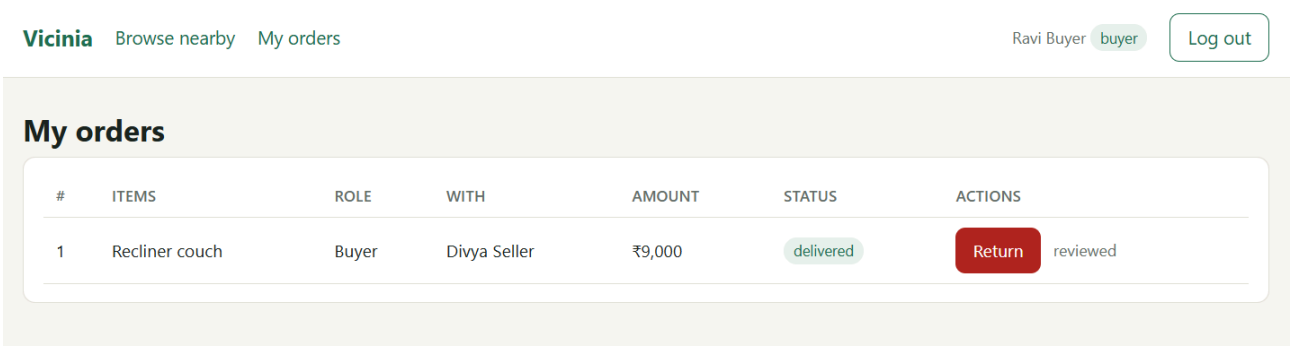
[Screenshot: listing detail with delivery estimate, pricing hint and trust path]



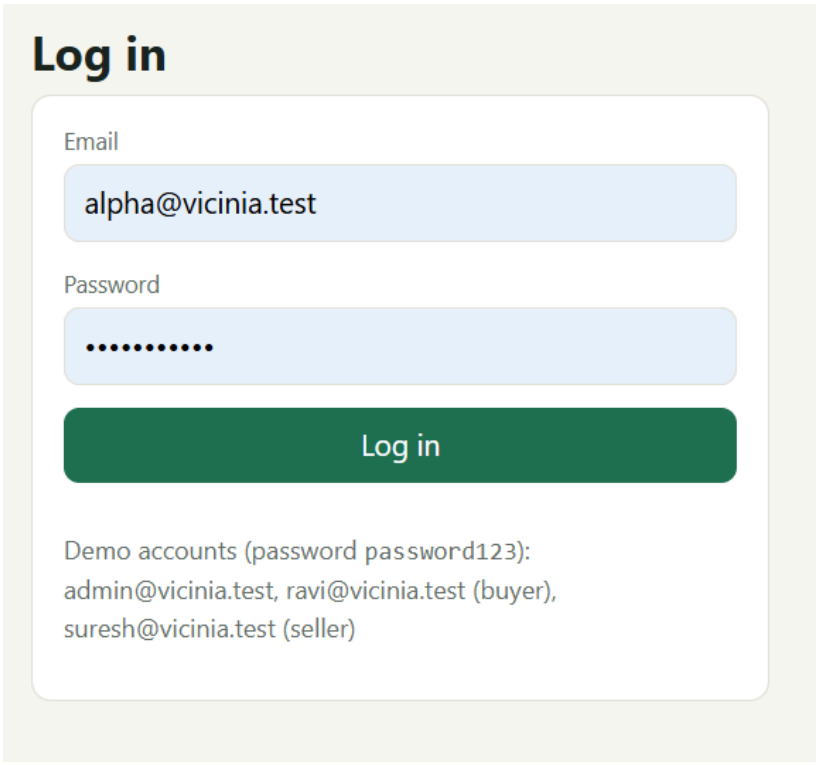
[Screenshot: seller dashboard and new listing page]

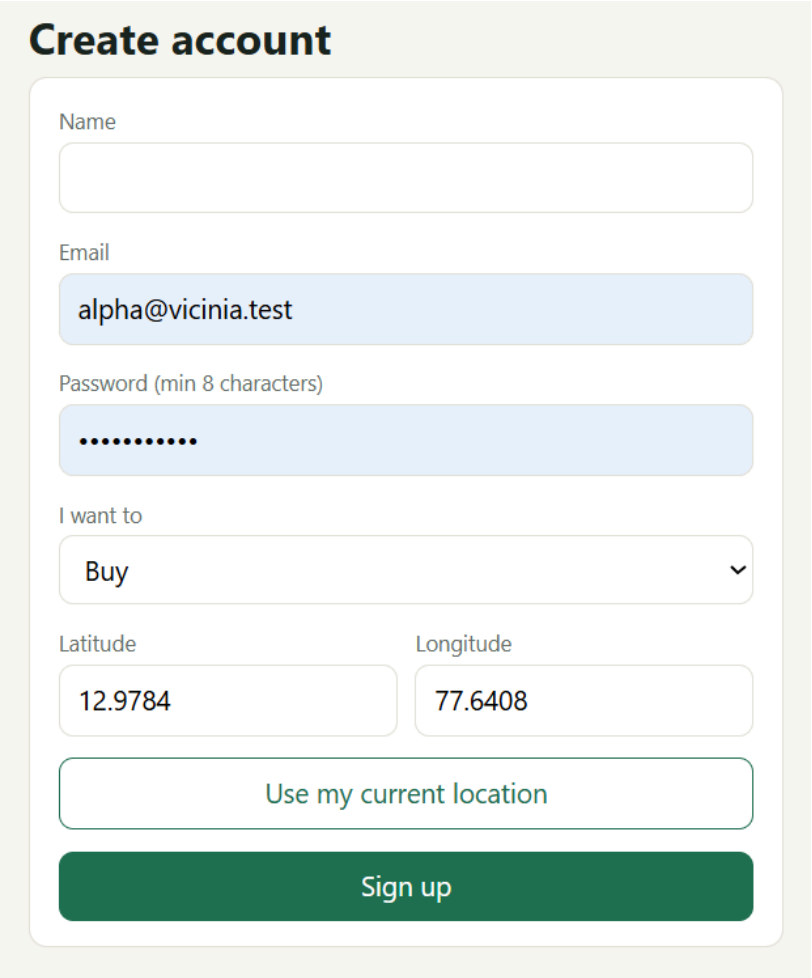


[Screenshot: my orders with deliver, return and review actions]

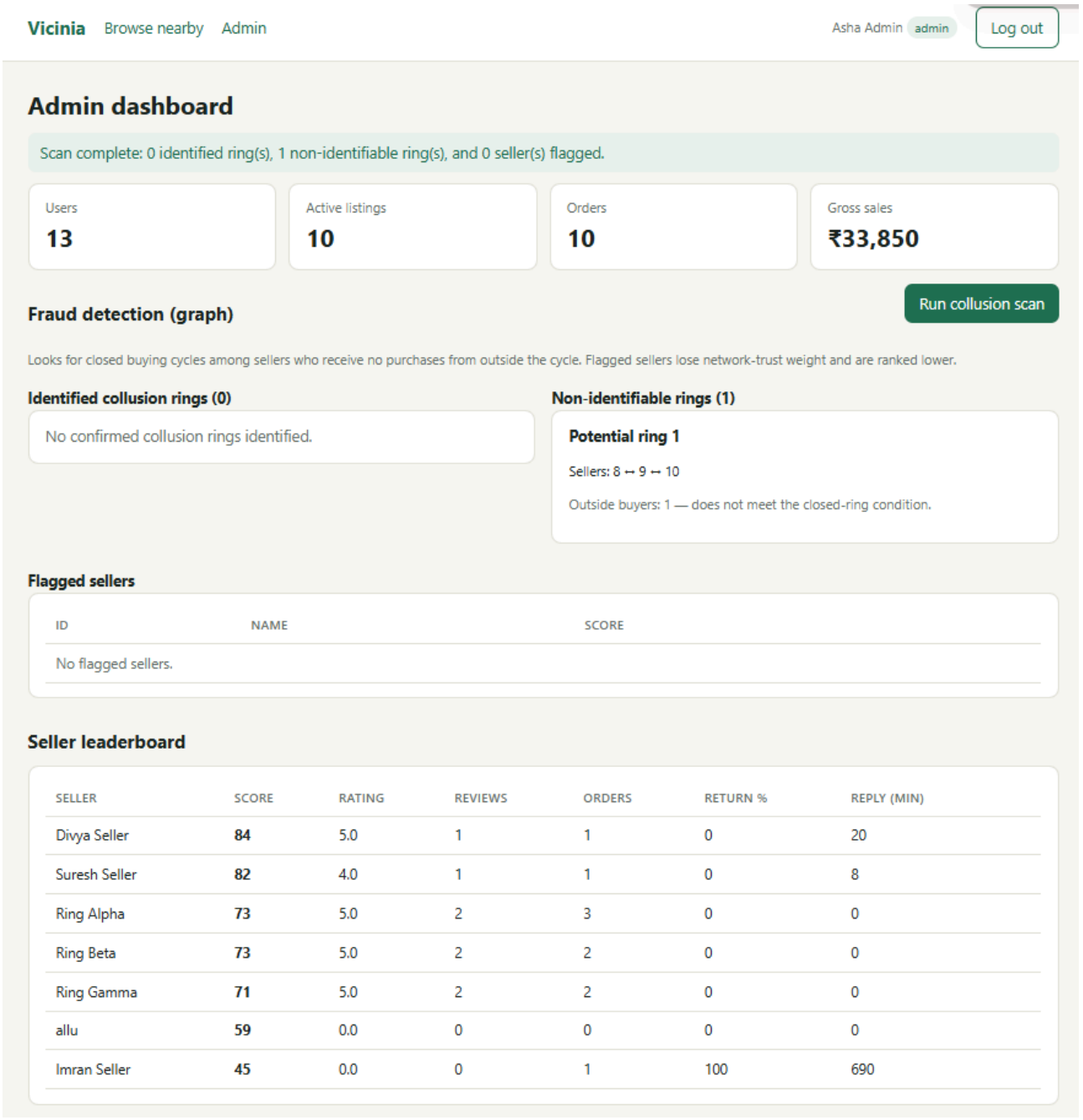


[Screenshot: login and register pages]





[Screenshot: admin dashboard with leaderboard, flagged sellers and Run collusion scan]


Vicinia
Browse nearby
Admin
Asha Admin
admin
Log out
Admin dashboard
Scan complete: 0 identified ring(s), 1 non-identifiable ring(s), and 0 seller(s) flagged.
Users
13
Active listings
10
Orders
10
Gross sales
₹33,850
Fraud detection (graph)
Run collusion scan
Looks for closed buying cycles among sellers who receive no purchases from outside the cycle. Flagged sellers lose network-trust weight and are ranked lower.
Identified collusion rings (0)
No confirmed collusion rings identified.
Non-identifiable rings (1)
Potential ring 1
Sellers: 8 ↔ 9 ↔ 10
Outside buyers: 1 — does not meet the closed-ring condition.
Flagged sellers
ID
NAME
SCORE
No flagged sellers.
Seller leaderboard
SELLER
SCORE
RATING
REVIEWS
ORDERS
RETURN %
REPLY (MIN)
Divya Seller
84
5.0
1
1
0
20
Suresh Seller
82
4.0
1
1
0
8
Ring Alpha
73
5.0
2
3
0
0
Ring Beta
73
5.0
2
2
0
0
Ring Gamma
71
5.0
2
2
0
0
allu
59
0.0
0
0
0
0
Imran Seller
45
0.0
0
1
100
690

Listings by category

CATEGORY	LISTINGS	SOLD	AVG PRICE
Electronics	7	6	₹764
Sofas	5	2	₹9,500
Phones	2	1	₹19,000
Laptops	2	1	₹13,750
Furniture	2	0	₹2,600
Appliances	1	0	₹14,000
Books	1	0	₹1,500

Team contribution

	Person A: Marketplace and Geo	**Person B: Trust, Identity and Intelligence**
Name / GitHub	[name, account]	[name, account]
PostgreSQL	users, categories, delivery_zones, listings, orders, order_items, price_history; indexes and views; purchase_listing, set_order_status, search_nearby, estimate_delivery, pricing_hint	reviews, messages, seller_metrics; compute_seller_score, refresh_seller_metrics, apply_graph_signals; review, order and user triggers
Neo4j	BOUGHT_FROM sync, recommendations	Trust edges, trust scores, collusion detection
API and pages	listings, categories, orders, dashboard, recommendations; /, /listing/[id], /sell, /orders	auth, reviews, trust, admin; /login, /register, /admin
Commits	[number]	[number]

Evaluation Rubrics

#	Criteria	Distribution	Status
1	Database Design	ER diagram , relational schema with keys, relationships, and constraints	Done
2	Database Creation	SQL database implementation - DDL with constraints , NoSQL/Vector database creation , Technology justification	Done
3	SQL Operations and Queries	DML operations , Nested/correlated/Recursive SQL queries, Join SQL queries	Done
4	Advanced Database Concepts	Advanced database concepts (Procedure/function, trigger, CTE) , NoSQL/Vector feature RUD	Done
5	Web Application and Database Integration	Next.js interface , Backend/API and database integration	Done
6	Authentication and Authorization	Authentication implementation (front end and backend) and minimum two roles and role-based access control in database	Done
7	Documentation, Git/GitHub, Demonstration, Testing, and Originality	Documentation, Git/GitHub , Originality and individual contribution	Pending: screenshots, names, commit counts

GITHUB LINK Spoorthi-Poornachandra/Vicinia---DBMS-Mini-Project: Hyperlocal marketplace: PostGIS spatial search + Neo4j trust graph, built with Next.js. DBMS Level 3 project, PES University.